# Grammar-Based Differential Fuzzing of Hugging Face Chatbots

**[Author 1][1], [Author 2][1], [Author 3][1], [Author 4][1]**

[1] [Department – University]
[City – State – Brazil]

[{author1,author2,author3,author4}@university.br]

***Abstract.*** *[Pending: filled when the experiments finish.]*

## 1. Introduction

Chat models built on large language models (LLMs) are now components of ordinary software: they answer questions, transform text, and produce structured output that other programs parse. Hundreds of open chat models are distributed through the Hugging Face Hub, many of them small enough to run on a laptop. Like any component, they need testing, but they are hard to test with classical techniques. They rarely crash, so a crash oracle reveals little; their input language is natural language, which has no parser to fuzz; and for most prompts there is no single correct output, which makes the test-oracle problem acute [Barr et al. 2015].

Fuzzing research offers two tools for exactly these obstacles. Grammar-based fuzzing generates large numbers of structured, valid inputs from a grammar [Holler et al. 2012], and grammar coverage can guide generation when the internals of the system under test cannot be observed [Havrikov and Zeller 2019]. Differential testing replaces a missing specification by comparing several implementations on the same input [McKeeman 1998], and metamorphic testing replaces it by comparing a system's outputs on related inputs [Chen et al. 1998]. Yet existing LLM fuzzers target jailbreaks rather than functional failures [Yu et al. 2023, Yao et al. 2024], and metamorphic frameworks for LLMs compare each model only against itself [Hyun et al. 2024, Cho et al. 2025].

In this project we ask whether combining these techniques yields an effective way to find functional failures in open chat models. We built `chatfuzz`, a fuzzer whose context-free grammar generates *verifiable* prompts: arithmetic, letter counting, string reversal, JSON formatting, verbatim echo of hard Unicode text, and free-form requests. Each derivation carries its expected answer and can be re-derived as paraphrases. The same prompts are sent to several chat models, so three oracles apply at once: correctness against ground truth, agreement across models, and invariance across paraphrases.

This paper makes the following contributions:

1. A grammar-based, coverage-guided fuzzer for chat models that generates paraphrase families with built-in ground truth, supporting local Hugging Face models and the Hugging Face Inference Providers API.

2. An analysis that combines ground-truth, differential, and metamorphic oracles and reports which model is wrong when models disagree, as well as prompts on which all models agree on a wrong answer.

3. An empirical evaluation on three small open chat models (Qwen2.5-0.5B-Instruct, SmolLM2-360M-Instruct, and TinyLlama-1.1B-Chat), including an ablation of coverage guidance and a manual check of oracle reliability.

Section 2 reviews related work, Section 3 states the research questions, Sections 4 and 5 describe the method and the experimental setup, Section 6 reports the results, Section 7 discusses them, Section 8 covers threats to validity, and Section 9 concludes.

## 2. Background and Related Work

Our approach combines four lines of work that have developed largely in separate communities: grammar-based fuzzing, differential testing, metamorphic testing, and the testing of LLMs.

### 2.1. Grammar-Based Fuzzing and Grammar Coverage

Grammar-based generation dates back to Purdom's sentence generator for testing parsers [Purdom 1972]. Modern grammar-based fuzzers such as LangFuzz [Holler et al. 2012], Skyfire [Wang et al. 2017], Nautilus [Aschermann et al. 2019], Superion [Wang et al. 2019], and Gramatron [Srivastava and Payer 2021] derive syntactically valid inputs for interpreters and parsers and rely on crash or sanitizer oracles. Nautilus, the closest precedent for coverage-guided grammar fuzzing, is guided by code coverage. When code coverage is unavailable, coverage of the input grammar itself can serve as an adequacy criterion: Havrikov and Zeller [2019] formalized $k$-path coverage, and its journal extension showed that input-structure coverage correlates with code coverage [Havrikov et al. 2022]. The `GrammarCoverageFuzzer` of *The Fuzzing Book* [Zeller et al. 2024] implements the 1-path variant by preferring uncovered expansions and is the direct basis of our generator. Probabilistic grammars offer an alternative way to steer generation [Soremekun et al. 2022].

All of these fuzzers target formal languages. None derives natural-language prompts, and none uses grammar coverage as the only feedback available against a black-box generative model.

### 2.2. Differential Testing

Differential testing runs one input through several systems that should behave alike and treats divergence as a failure [McKeeman 1998], extending the earlier pseudo-oracle idea [Davis and Weyuker 1981]. Csmith made it a standard method for compilers by generating only valid C programs, so that every disagreement is meaningful [Yang et al. 2011]. Coverage guidance and differential oracles have been combined before, for example for JVM implementations [Chen et al. 2016]. In machine learning, DeepXplore used disagreement among several deep networks as an oracle that needs no labels [Pei et al. 2017], and CRADLE compared deep-learning library backends [Pham et al. 2019]. Re-

cent LLM-related differential work mostly uses the LLM as a *generator* of tests for other software, as in TitanFuzz [Deng et al. 2023], rather than as the system under test.

Unlike compilers, chat LLMs share no formal specification, so a disagreement is only a signal. Work on hallucination detection uses cross-sample and cross-model consistency in a similar way [Manakul et al. 2023, Zhang et al. 2023], but also warns that models can agree on a wrong answer [Zhang et al. 2023]. Our verifiable prompts address both points: ground truth identifies which model is wrong and exposes cases where all models are wrong.

### 2.3. The Test-Oracle Problem and Metamorphic Testing

Free-form chatbots are "non-testable" programs in Weyuker's sense [Weyuker 1982]: there is no practical way to specify the correct output for every input. Barr et al. [2015] survey solutions to this oracle problem. Metamorphic testing [Chen et al. 1998] checks relations between the outputs of related inputs instead of individual outputs. In NLP, CheckList generates tests from templates and checks invariance under meaning-preserving changes [Ribeiro et al. 2020], and TransRepair found inconsistency bugs in machine translation without a human oracle [Sun et al. 2020]. Consistency under meaning-preserving rephrasing is poor in pretrained language models [Elazar et al. 2021], and single-prompt evaluation of LLMs is unreliable [Mizrahi et al. 2024, Errica et al. 2025]. Metamorphic testing frameworks for LLMs have appeared at software-testing venues, including METAL [Hyun et al. 2024], the large catalogue of metamorphic relations by Cho et al. [2025], and MORTAR for multi-turn dialogue systems [Guo et al. 2026]. Drowzee derives metamorphic test cases with known answers to detect factual hallucinations [Li et al. 2024].

These frameworks compare a model against itself: a violated relation shows inconsistency but not which answer is wrong. Our paraphrase relation is a CheckList-style invariance test, but the same grammar derivation also provides the expected answer and is sent to several models. Self-consistency decoding [Wang et al. 2023] votes over sampled *outputs* to improve accuracy, whereas we test invariance over paraphrased *inputs* to find failures.

### 2.4. Fuzzing and Testing LLMs and Chatbots

Existing LLM fuzzers target security. GPTFuzzer mutates jailbreak templates [Yu et al. 2023], FuzzLLM composes jailbreak prompts from fixed parts [Yao et al. 2024], and h4rm3l synthesizes attacks in a domain-specific language [Doumbouya et al. 2025]; their oracle is attack success. Earlier chatbot testing in software engineering addressed rule-based or recurrent dialogue systems [Bozic and Wotawa 2019, Liu et al. 2021]. Wang et al. [2024] survey the broader use of LLMs in software testing.

The NLP literature documents the functional weaknesses our grammar targets. IFEval evaluates instructions that code can verify [Zhou et al. 2023]. Formatting changes alone can change accuracy by up to 76 points [Sclar et al. 2024]. Models often fail at character-level manipulation such as spelling, counting, and reversal [Edman et al. 2024, Chai et al. 2024]; under-trained tokens can break rare Unicode strings [Land and Bartolo 2024]; and template-instantiated variants of math problems expose unstable reasoning

[Mirzadeh et al. 2025]. These results come from static benchmarks, mostly on large models.

### 2.5. Positioning

To our knowledge, no prior work combines a context-free prompt grammar with grammar-coverage guidance, a cross-model differential oracle, and a cross-paraphrase metamorphic oracle for open chat models. Each ingredient has a canonical precedent: the `GrammarCoverageFuzzer` for generation, Csmith and DeepXplore for the differential oracle, and CheckList for the invariance oracle. Our contribution is to combine them, with the grammar derivation itself supplying the ground truth that settles disagreements.

## 3. Research Questions and Objectives

Our objective is to assess whether grammar-based fuzzing combined with differential and metamorphic oracles is an effective way to reveal failures in open chat LLMs, and where the approach falls short. We address four research questions:

**RQ1 (Effectiveness).** How many failures, and of which kinds, does grammar-based fuzzing reveal in small open chat models from the Hugging Face Hub?

**RQ2 (Oracles).** What do the differential (cross-model) and metamorphic (cross-paraphrase) oracles add beyond oracles that rely on a known expected answer?

**RQ3 (Guidance).** Does coverage-guided grammar expansion improve grammar coverage and the diversity of revealed failures compared with uniform random expansion under the same budget?

**RQ4 (Oracle reliability).** How often do the heuristic oracles misjudge a response, as measured on a manually inspected sample?

RQ1 and RQ2 characterize what the approach finds, RQ3 evaluates a design choice of the generator, and RQ4 bounds the validity of the reported numbers.

## 4. Methodology

Our tool, `chatfuzz`, has four parts: a grammar-based prompt generator, chatbot clients, oracles, and an analysis stage that applies the differential and metamorphic relations. All code and unit tests are in the project repository.

### 4.1. Test Case Generation

Each test case starts from a *skeleton*: a task category and randomly drawn values, such as the operands of an arithmetic expression. Because the values are stored, every test case knows its expected answer. A context-free grammar then derives the wording of the request. Deriving the same skeleton several times yields *paraphrases* that must receive the same answer. The grammar has one start symbol per category (Table 1).

**Table 1. Task categories, example prompts generated by the grammar, and ground-truth oracles.**

| Category | Example prompt | Oracle |
|---|---|---|
| `arith` | *What is 4 multiplied by 101? Give only the numeric result.* | last integer in the reply equals the result |
| `count` | *How many times does the letter 'e' appear in the word 'referrer'?* | integer equals the true count |
| `reverse` | *Reverse the string 'NhFLOv2mp'. Reply with the result only.* | normalized reply equals the reversed string |
| `json` | *Return a JSON object with the keys "city" and "tags". Reply with JSON only.* | reply parses as a JSON object with the requested keys |
| `echo` | *Repeat exactly the following text, without any change:* followed by emoji, right-to-left or invisible characters | payload appears verbatim in the reply |
| `edge` | *Translate to Portuguese:* followed by a hard Unicode payload | no runtime error and a non-empty reply |

Operands mix one-digit, three-digit, six-digit, and negative numbers. Letters for `count` are drawn mostly from the word and sometimes from outside it, so that the expected count is zero. The `echo` and `edge` payloads include combining emoji, right-to-left script, zero-width characters, ligatures, markup, and a 300-character repetition.

### 4.2. Coverage-Guided Expansion

When a nonterminal has several expansions, the generator chooses an expansion that has not yet been used with probability $b$ (`coverage_bias`, 0.8 by default), and a uniformly random one otherwise. This follows the grammar coverage fuzzer of Zeller et al. [2024] and corresponds to 1-path grammar coverage [Havrikov and Zeller 2019]. Grammar coverage is the fraction of all (nonterminal, expansion) pairs used so far. Paraphrases of a skeleton must differ in their words, not only in whitespace.

### 4.3. Chatbot Clients

Models are addressed as `local:<model>` or `api:<model>`. Local models are downloaded from the Hugging Face Hub and run with the `transformers` library, using the model's own chat template and greedy decoding, so that each prompt has a single deterministic reply. The API client sends OpenAI-compatible chat completion requests to the Hugging Face Inference Providers router. Runtime errors, timeouts, and malformed replies are recorded as data rather than raised. A reply that stops at the token limit is flagged as truncated.

### 4.4. Oracles and Analysis

The analysis applies three kinds of oracle:

1. **Ground-truth oracles** check each reply against the expected answer of its category (Table 1) and extract a *canonical answer*: an integer, a normalized string, the sorted

key list of a JSON object, or a Boolean for `echo`.

2. **Differential oracle.** The same prompt is sent to every model. Different canonical answers produce a *differential disagreement*, which names the models that are wrong when the expected answer is known; a model that fails at runtime while others reply produces a *differential error*. When all models give the same wrong answer, the analysis reports a *consensus-wrong* prompt, a case that a purely differential oracle cannot detect [Zhang et al. 2023].

3. **Metamorphic oracle.** For each model, the paraphrases of one skeleton must yield the same canonical answer; otherwise the tool reports a *metamorphic inconsistency*.

The tool reports, per model, the pass rate by category, metamorphic consistency, pairwise disagreement between models, and the number of runtime errors and truncated replies.

## 5. Experimental Setup

**Models under test.** We selected three small instruction-tuned chat models from different families, so that differences between them are not artifacts of a shared training pipeline (Table 2). All models ran locally with greedy decoding and at most 256 new tokens. We did not use the API client in the experiments because no Hugging Face API token was available in the execution environment.

**Table 2. Models under test.**

| Model | Parameters | Revision | Reference |
|---|---|---|---|
| `Qwen/Qwen2.5-0.5B-Instruct` | 0.5 B | `7ae5576` | Qwen Team [2024] |
| `HuggingFaceTB/SmolLM2-360M-Instruct` | 360 M | `a10cc15` | Ben Allal et al. [2025] |
| `TinyLlama/TinyLlama-1.1B-Chat-v1.0` | 1.1 B | `fe8a4ea` | Zhang et al. [2024] |

**Environment.** A Linux cloud container with 4 CPU cores and 15 GB of RAM, no GPU, Python 3.13, PyTorch 2.14.1, and `transformers` 5.21.0.

**Main campaign (RQ1, RQ2, RQ4).** The generator produced 60 skeletons, 10 per category, each derived as 3 paraphrases, for 180 prompts per model and 540 model responses in total (grammar seed 0, $b = 0.8$). Every model received the same prompts.

**Ablation 1: grammar coverage without models (RQ3).** For 30 grammar seeds, we generated 120 prompts with uniform random expansion ($b = 0$) and with coverage-guided expansion ($b = 0.8$), and recorded grammar coverage after each prompt.

**Ablation 2: failure diversity with models (RQ3).** For grammar seeds 0, 1, and 2 and both values of $b$, we ran 30 skeletons with one wording each on the two faster models, Qwen2.5 and SmolLM2. We compare the number of failures, the number of distinct grammar rules that appear in failing prompts, and the number of distinct failure signatures (model, category, oracle verdict, and top-level template).

**Oracle validation (RQ4).** We drew a stratified random sample of the main campaign's graded responses, balanced across categories and models, and compared each

automatic verdict with a manual judgment of whether the reply answered the request correctly.

**Metrics.** Pass rate per model and category; number of findings of each type; metamorphic consistency, the share of skeletons for which a model gave the same canonical answer to all paraphrases; pairwise disagreement between models on comparable prompts; consensus-wrong prompts; and grammar coverage.

## 6. Results

[Pending: filled when the experiments finish.]

## 7. Discussion

[Pending: filled when the experiments finish.]

## 8. Threats to Validity and Limitations

**Construct validity.** The ground-truth oracles rely on heuristics: the numeric oracle takes the last integer in the reply, and the reversal oracle strips quotes and punctuation. A verbose but correct reply can therefore be judged wrong, and vice versa; RQ4 measures this error on a sample. The differential oracle compares canonical answers, so two semantically equivalent replies with different surface forms can count as a disagreement. The `edge` category only checks for runtime errors and empty replies, so it cannot reveal wrong answers.

**Internal validity.** Greedy decoding makes each local model deterministic for a given software stack, but results may change with other library versions or hardware. Replies cut at the 256-token limit are flagged, and failures among them are reported separately rather than discarded. The oracle validation covers a sample, not all responses, and the manual judgments were made with AI assistance (see the AI use disclosure).

**External validity.** We tested three small models (360 M to 1.1 B parameters) running locally. Larger models, models served through the API, and other providers may behave very differently, and our pass rates should not be read as statements about chat LLMs in general. The prompts are in English and cover six task categories; they do not represent open-ended conversation.

**Conclusion validity.** The main campaign uses one grammar seed, and the failure-diversity ablation uses three, so differences between conditions in Ablation 2 are indicative rather than statistically established. Ablation 1, which needs no models, uses 30 seeds.

**Naturalness of generated prompts.** Grammar-derived prompts are well formed but templated; real users phrase requests more freely. The grammar's coverage of phrasings is limited to the expansions we wrote.

## 9. Conclusion

[Pending: filled when the experiments finish.]

## AI Use Disclosure

During the preparation of this work, the authors used Claude (Anthropic), through Claude Code, to implement the fuzzing tool and its unit tests, run the experiments, search and summarize related work, judge the oracle-validation sample, and draft this paper. All numerical results come from the runs described in Section 5 and can be reproduced with the scripts in the project repository. The authors are responsible for reviewing the content; bibliographic entries marked as unverified must be checked against the original sources before submission.

## References


Aschermann, C., Frassetto, T., Holz, T., Jauernig, P., Sadeghi, A.-R., and Teuchert, D. (2019). NAUTILUS: Fishing for deep bugs with grammars. In *Network and Distributed System Security Symposium (NDSS)*.

Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., and Yoo, S. (2015). The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525.

Ben Allal, L. et al. (2025). SmolLM2: When smol goes big. arXiv:2502.02737.

Bozic, J. and Wotawa, F. (2019). Testing chatbots using metamorphic relations. In *Testing Software and Systems (ICTSS)*, LNCS. Springer.

Chai, Y., Fang, Y., Peng, Q., and Li, X. (2024). Tokenization falling short: On subword robustness in large language models. In *Findings of the Association for Computational Linguistics: EMNLP*.

Chen, T. Y., Cheung, S. C., and Yiu, S. M. (1998). Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology. Available as arXiv:2002.12543.

Chen, Y., Su, T., Sun, C., Su, Z., and Zhao, J. (2016). Coverage-directed differential testing of JVM implementations. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.

Cho, S., Ruberto, S., and Terragni, V. (2025). Metamorphic testing of large language models for natural language processing. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 174–186.

Davis, M. D. and Weyuker, E. J. (1981). Pseudo-oracles for non-testable programs. In *Proceedings of the ACM '81 Conference*.

Deng, Y., Xia, C. S., Peng, H., Yang, C., and Zhang, L. (2023). Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*.

Doumbouya, M. K. B. et al. (2025). h4rm3l: A language for composable jailbreak attack synthesis. In *International Conference on Learning Representations (ICLR)*. arXiv:2408.04811. Author list unverified.

Edman, L., Schmid, H., and Fraser, A. (2024). CUTE: Measuring LLMs' understanding of their tokens. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Elazar, Y., Kassner, N., Ravfogel, S., Ravichander, A., Hovy, E., Schütze, H., and Goldberg, Y. (2021). Measuring and improving consistency in pretrained language models. *Transactions of the Association for Computational Linguistics*, 9.

Errica, F., Sanvito, D., Siracusano, G., and Bifulco, R. (2025). What did I do wrong? quantifying LLMs' sensitivity and consistency to prompt engineering. In *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Guo, G., Aleti, A., Neelofar, N., Tantithamthavorn, C., Qi, Y., and Chen, T. Y. (2026). MORTAR: Multi-turn metamorphic testing for LLM-based dialogue systems. *IEEE Transactions on Software Engineering*.

Havrikov, N., Kampmann, A., and Zeller, A. (2022). From input coverage to code coverage: Systematically covering input structure with k-paths. *ACM Transactions on Software Engineering and Methodology*. DOI unverified.

Havrikov, N. and Zeller, A. (2019). Systematically covering input structure. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 189–199.

Holler, C., Herzig, K., and Zeller, A. (2012). Fuzzing with code fragments. In *21st USENIX Security Symposium*, pages 445–458.

Hyun, S., Guo, M., and Babar, M. A. (2024). METAL: Metamorphic testing framework for analyzing large-language model qualities. In *IEEE Conference on Software Testing, Verification and Validation (ICST)*.

Land, S. and Bartolo, M. (2024). Fishing for Magikarp: Automatically detecting undertrained tokens in large language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Li, N., Li, Y., Liu, Y., Shi, L., Wang, K., and Wang, H. (2024). Drowzee: Metamorphic testing for fact-conflicting hallucination detection in large language models. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA2).

Liu, Z., Feng, Y., and Chen, Z. (2021). DialTest: Automated testing for recurrent-neural-network-driven dialogue systems. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*.

Manakul, P., Liusie, A., and Gales, M. (2023). SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

McKeeman, W. M. (1998). Differential testing for software. *Digital Technical Journal*, 10(1):100–107. Partially verified.

Mirzadeh, I., Alizadeh, K., Shahrokhi, H., Tuzel, O., Bengio, S., and Farajtabar, M. (2025). GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models. In *International Conference on Learning Representations (ICLR)*. arXiv:2410.05229.

Mizrahi, M., Kaplan, G., Malkin, D., Dror, R., Shahaf, D., and Stanovsky, G. (2024). State of what art? A call for multi-prompt LLM evaluation. *Transactions of the Association for Computational Linguistics*, 12.

Pei, K., Cao, Y., Yang, J., and Jana, S. (2017). DeepXplore: Automated whitebox test-

ing of deep learning systems. In *ACM Symposium on Operating Systems Principles (SOSP)*.

Pham, H. V., Lutellier, T., Qi, W., and Tan, L. (2019). CRADLE: Cross-backend validation to detect and localize bugs in deep learning libraries. In *IEEE/ACM International Conference on Software Engineering (ICSE)*.

Purdom, P. (1972). A sentence generator for testing parsers. *BIT Numerical Mathematics*, 12.

Qwen Team (2024). Qwen2.5 technical report. arXiv:2412.15115.

Ribeiro, M. T., Wu, T., Guestrin, C., and Singh, S. (2020). Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4902–4912.

Sclar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. (2024). Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.11324.

Soremekun, E., Pavese, E., Havrikov, N., Grunske, L., and Zeller, A. (2022). Inputs from hell: Learning input distributions for grammar-based test generation. *IEEE Transactions on Software Engineering*, 48(4).

Srivastava, P. and Payer, M. (2021). Gramatron: Effective grammar-aware fuzzing. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 244–256.

Sun, Z., Zhang, J. M., Harman, M., Papadakis, M., and Zhang, L. (2020). Automatic testing and improvement of machine translation. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 974–985.

Wang, J., Chen, B., Wei, L., and Liu, Y. (2017). Skyfire: Data-driven seed generation for fuzzing. In *IEEE Symposium on Security and Privacy (S&P)*.

Wang, J., Chen, B., Wei, L., and Liu, Y. (2019). Superion: Grammar-aware greybox fuzzing. In *IEEE/ACM International Conference on Software Engineering (ICSE)*.

Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., and Wang, Q. (2024). Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*.

Wang, X. et al. (2023). Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*. arXiv:2203.11171.

Weyuker, E. J. (1982). On testing non-testable programs. *The Computer Journal*, 25(4):465–470.

Yang, X., Chen, Y., Eide, E., and Regehr, J. (2011). Finding and understanding bugs in C compilers. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.

Yao, D., Zhang, J., Harris, I. G., and Carlsson, M. (2024). FuzzLLM: A novel and universal fuzzing framework for proactively discovering jailbreak vulnerabilities in large

language models. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4485–4489.

Yu, J., Lin, X., Yu, Z., and Xing, X. (2023). GPTFUZZER: Red teaming large language models with auto-generated jailbreak prompts. arXiv:2309.10253.

Zeller, A., Gopinath, R., Böhme, M., Fraser, G., and Holler, C. (2024). Grammar coverage. In The Fuzzing Book. CISPA Helmholtz Center for Information Security. `https://www.fuzzingbook.org/html/GrammarCoverageFuzzer.html`. Accessed: 2026-10-29.

Zhang, J., Li, Z., Das, K., Malin, B., and Kumar, S. (2023). SAC3: Reliable hallucination detection in black-box language models via semantic-aware cross-check consistency. In *Findings of the Association for Computational Linguistics: EMNLP*.

Zhang, P., Zeng, G., Wang, T., and Lu, W. (2024). TinyLlama: An open-source small language model. arXiv:2401.02385.

Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L. (2023). Instruction-following evaluation for large language models. arXiv:2311.07911.